Update my existing FlashDrop Android application built with Kotlin, Jetpack Compose, and Firebase. Implement a complete, functional Edit Profile feature for all three user roles: Customer, Shopkeeper, and both Main Admins.

IMPORTANT: Inspect the existing project and integrate this feature into the current app. Do not create a separate demo app or break existing functionality.

1. CUSTOMER – EDIT PROFILE

Add an “Edit Profile” option inside the Customer's Profile & Settings page.

Allow customers to:

- Edit their full name.
- Change their profile photo by selecting an image from their phone gallery or taking a new photo using the camera, where supported.
- Edit their mobile number.
- Edit their email address, subject to Firebase Authentication verification requirements.
- Edit their delivery addresses, including house number, street, area, city, state, PIN code, and a landmark.
- Add multiple delivery addresses and select a default address.
- Update their profile information using a clearly visible “Save Changes” button.
- Cancel editing without saving changes.

Display the updated name and profile photo throughout the Customer section wherever applicable, including Profile, Checkout, and Order Details.

Do not allow customers to change their Firebase UID or access another customer's profile.

2. SHOPKEEPER – EDIT SHOP PROFILE

Add an “Edit Shop Profile” option inside the Shopkeeper's dashboard or Profile section.

Allow shopkeepers to edit permitted information:

- Shop name.
- Shop profile photo or logo.
- Shop banner image.
- Shop mobile number.
- Shop address and location.
- Shop opening and closing hours.
- Shop description.
- Shop category.
- GST number, with appropriate verification and restrictions.

Allow shopkeepers to update their shop information using a “Save Changes” button.

IMPORTANT SHOPKEEPER PERMISSIONS:

- Shopkeepers must not be able to change their Shop ID, assigned account ID, approval status, or permissions.
- Shopkeepers must not be able to promote themselves to Admin.
- Changes to sensitive information, such as the GST number or verified mobile number, must require appropriate verification or Admin approval.
- Shopkeepers can edit only their own shop profile.
- Preserve existing product management, inventory, order management, and earnings functionality.
- Do not allow shopkeepers to edit another shopkeeper's profile or shop information.

3. MAIN ADMIN – EDIT ADMIN PROFILE

Add an “Edit Admin Profile” option in the Main Admin Portal for both authorized Main Admins.

Authorized Main Admin accounts:

- Pranjal Rajpoot — existing Main Admin account.
- Chanchal Didi — existing Main Admin account.

Both Main Admins must have equal administrative permissions.

Allow each Main Admin to:

- Edit their display name, where permitted.
- Update their profile photo.
- Update their mobile number after verification.
- Update other permitted personal profile information.
- Save or cancel profile changes.

SECURITY REQUIREMENTS:

- Neither Admin can edit the other Admin's personal profile without appropriate authorization.
- Neither Admin can change their role or grant themselves additional privileges through the Edit Profile screen.
- Keep Admin account IDs and authentication credentials secure.
- Never display passwords in the app.
- Do not hardcode passwords into the UI or expose them in logs.
- Preserve both Main Admins' equal permissions for shopkeeper management, delivery charge settings, payment settings, products, orders, and other existing administrative functions.

4. PROFILE PHOTO UPLOAD

Implement functional profile image selection and upload.

Requirements:

- Support image selection from the phone gallery.
- Support camera capture where available and permission is granted.
- Display an image preview before saving.
- Allow users to replace their existing profile photo.
- Upload images to Firebase Storage if it is configured in the existing project.
- Store the resulting image URL in the appropriate Firebase profile document.
- Handle upload failures with a clear error message.
- Show a default avatar if the user has not uploaded a photo.
- Resize or compress large images appropriately before upload.

Do not store large image files directly inside Firestore documents.

5. FIREBASE DATABASE INTEGRATION

Inspect the existing Firebase database structure before making changes.

Store profile information in the appropriate existing collections or documents, maintaining a structure similar to:

- users/{uid} — Customer profile information.
- shops/{shopId} — Shopkeeper and shop profile information.
- admins/{adminUid} — Admin profile information, if this matches the existing architecture.

These are suggested paths, not instructions to duplicate existing collections. Reuse the existing schema wherever possible.

Requirements:

- Associate each profile with the correct authenticated Firebase UID or authorized Shop ID.
- Save profile changes to Firebase Firestore.
- Load existing profile information when the Edit Profile screen opens.
- Show a loading indicator while data is loading or saving.
- Show success and error messages.
- Refresh the relevant screens after a successful update.
- Handle offline conditions and Firebase errors gracefully.
- Prevent unauthorized users from modifying another person's profile.

6. FIREBASE SECURITY RULES

Implement or update secure Firebase Security Rules appropriate to the existing database structure.

- Customers can read and update only the permitted fields of their own profile.
- Shopkeepers can update only permitted fields of their own shop profile.
- Shopkeepers cannot approve themselves, change their assigned Shop ID, or change their role.

- Admin privileges must be verified using a trusted authorization mechanism, such as server-managed custom claims or another secure existing authorization system.
- Both authorized Main Admins must have equivalent permissions.
- Do not trust a role value supplied by the client.
- Prevent unauthorized users from changing account roles, approval statuses, permissions, ownership, or other protected fields.
- Restrict profile image uploads to authorized users and appropriate file types and sizes.
- Never make private user profiles publicly writable.
- Do not replace existing security rules blindly. Merge the required changes with existing rules while preserving legitimate app functionality.

7. UI AND DESIGN

Follow FlashDrop's existing design system:

- Existing FlashDrop logo and branding.
- Green and orange color theme.
- Clean, modern Android interface.
- Rounded cards and buttons.
- Responsive Jetpack Compose layouts for different phone sizes.
- Proper text spacing and scrolling.
- Clear loading, success, validation, and error states.

Create a dedicated Edit Profile screen for each role, using shared reusable UI components where appropriate.

Suggested screen structure:

1. Profile photo with an Edit Photo button.
2. Editable profile fields.
3. Read-only account ID and role information.
4. Save Changes button.
5. Cancel button.

Use suitable validation messages for missing names, invalid phone numbers, invalid email addresses, and incomplete addresses.

8. IMPORTANT EXISTING FUNCTIONALITY

Do not break:

- Google OAuth and Firebase Authentication.
- Persistent login sessions.
- Guest browsing and mandatory sign-in before ordering.
- Customer cart, checkout, and payment.
- Order history and delivery tracking.
- Shopkeeper registration and separate Shopkeeper Sign-In.
- Main Admin login and authorization.

- Admin approval of shopkeepers.
- Product management.
- Delivery charges and free-delivery settings.
- Existing Firebase database records.

If an email or phone number is managed by Firebase Authentication, update it through the appropriate authenticated Firebase flow and require reauthentication or verification when necessary. Do not merely change the Firestore display field and claim the authentication credential has changed.

9. IMPLEMENTATION AND TESTING

Implement actual working code, not just a visual mock-up.

Before finishing:

- Inspect the current authentication, database, and navigation architecture.
- Reuse existing models, repositories, and ViewModels wherever possible.
- Implement the Edit Profile screens and connect them to Firebase.
- Verify that each role can edit only its permitted fields.
- Verify that updated information appears throughout the relevant screens.
- Test successful saves, cancelled edits, invalid input, upload failures, and network errors.
- Confirm that Admin permissions, Shopkeeper permissions, and Customer access remain separate.
- Fix compilation errors introduced by the changes.

Finally, summarize the files changed, explain any Firebase configuration or Security Rules updates required, and provide the steps to test the feature on a real Android phone.